RAS API PLDM mapping

DMTF Manageability Workshop

Justin King (AMD), Samer El-Haj-Mahmoud (Arm), Janusz Jurski (Intel)

Disclaimer

- The information in this presentation represents a snapshot of work in progress within the DMTF.
- This information is subject to change without notice. The standard specifications remain the normative reference for all information.
- For additional information, see the DMTF website.
- This information is a summary of the information that will appear in the specifications. See the specifications for further details.

Overview

- Latest OCP RAS API v0.95rc1WIP: OCP_RAS_API_WIP - OCP HWMgt RAS API - Google Drive
  - Defines RAS API Endpoint and RAS API Initiator
  - Defines CPAD data structured
    - Equivalent to CPERs but for action requests

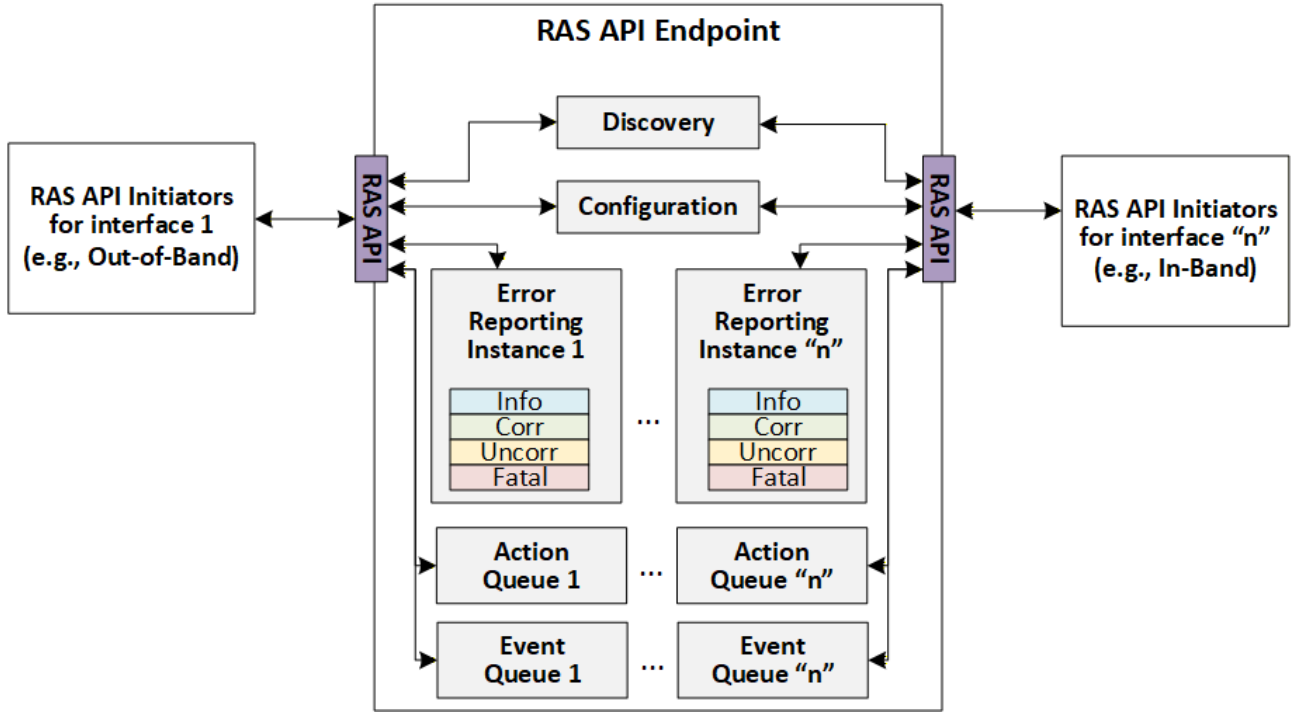
- PMCI ongoing effort to define RAS API PLDM model.
  - Requirements deck:
    - https://www.dmtf.org/sites/default/files/RAS_API_mapping_to_PLDM_requirements_WIP.pdf
  - Modeling to be defined in the DSP2094 non-normative document.

- This presentation walks through the DSP2094 (RAS API PLDM modeling) requirements and explores the RAS API modeling proposed directions and opens.

Anatomy of a RAS API EP

- A RAS API EP may have multiple interfaces.
- Interfaces are independently operated, allowing in-band and out-of-band flows to coexist.
- An EP contains per interface:
  - 1 action queue.
  - 1 event queue.
  - Up to 4 Error reporting queues (1 for each severity).

- Queues are managed independently.
- Each queue has a maximum size.



RAS API PLDM mapping document requirements

- Goal: enable interoperability of RAS API Endpoints (EP) and Initiators, that communicate over PLDM.
  - Informative for DMTF, likely normative for other orgs, like OCP.
  - Spec is an informational whitepaper (DSP2094), not a new PLDM type.
- Define RAS API EP discovery
  - PDR modeling of Error and Action files
    - Define mode and usage pattern of Error and Action files.
- Define RAS API EP configuration
  - Dependency on the OCP RAS API spec defining detailed EP configuration in future releases.
- Model RAS API queues using PLDM type 2 (monitoring and control) and type 7 (file transfer).

RAS API PLDM mapping document requirements 2

- Define RAS API primitive mapping to PLDM command/response messages
  - Read error records
    - Discover error record size
    - Define error record notifications
  - Clear error records (optional command for the PLDM modelling)
  - Request RAS Action
  - Retrieve Action status

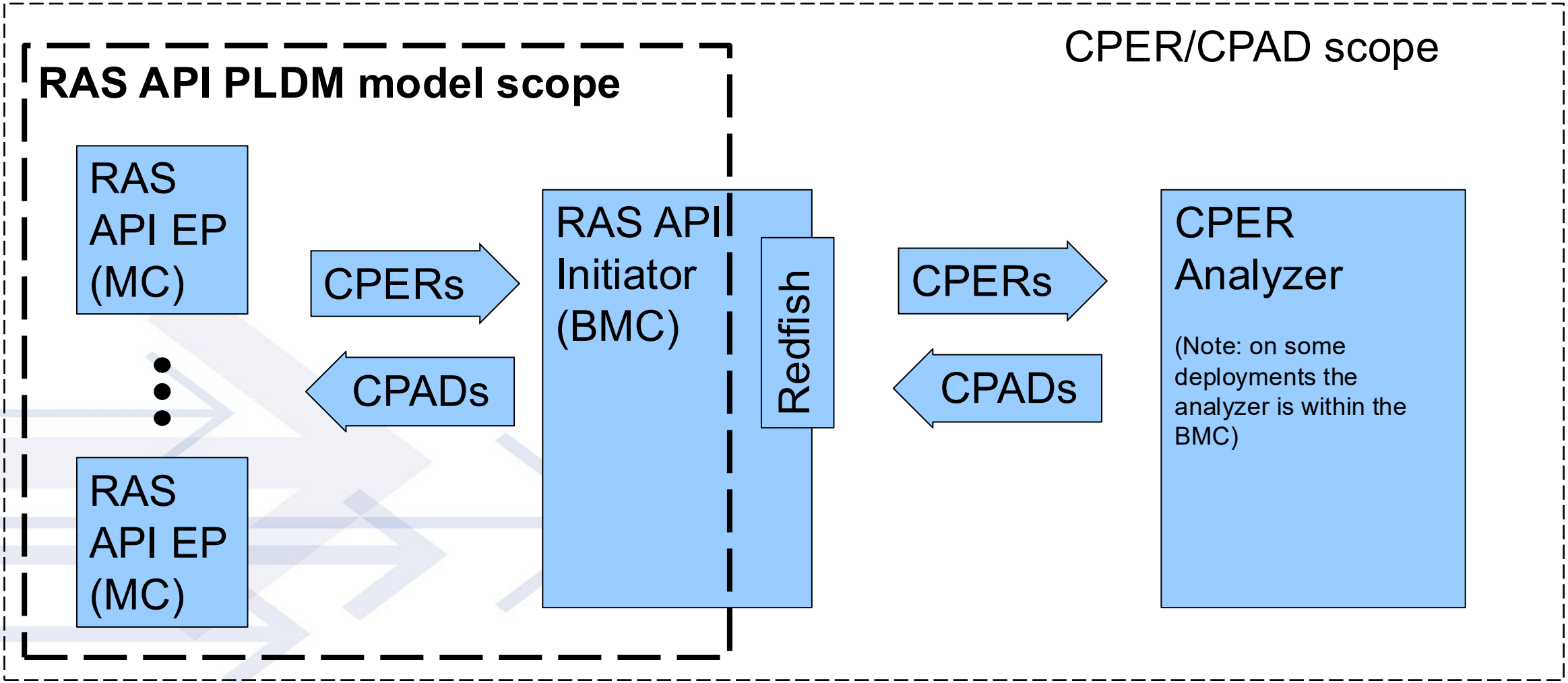
- Ensure PLDM RAS API Initiator can expose the necessary information and generate the relevant events for a Redfish implementation of a RAS API Endpoint.

Dependency on other DMTF standards

- PLDM for File Transfer Protocol (DSP0242)
  - FileClassification definitions in support of RAS API error and action files.
  - Dependency on DfWrite (to be released publicly in v1.1.0) for action requests.
- PLDM Platform Descriptor Record Specification (DSP0251)
  - Extend FileClassification in support of RAS API error and action file PDR description.
- PLDM for Platform Monitoring and Control (DSP0248 1.4.0)
  - Dependency on StructuredAttribute (to be released publicly), if used for “clear CPER” effecter and RAS API EP configuration get/set.

MODELING DETAILS

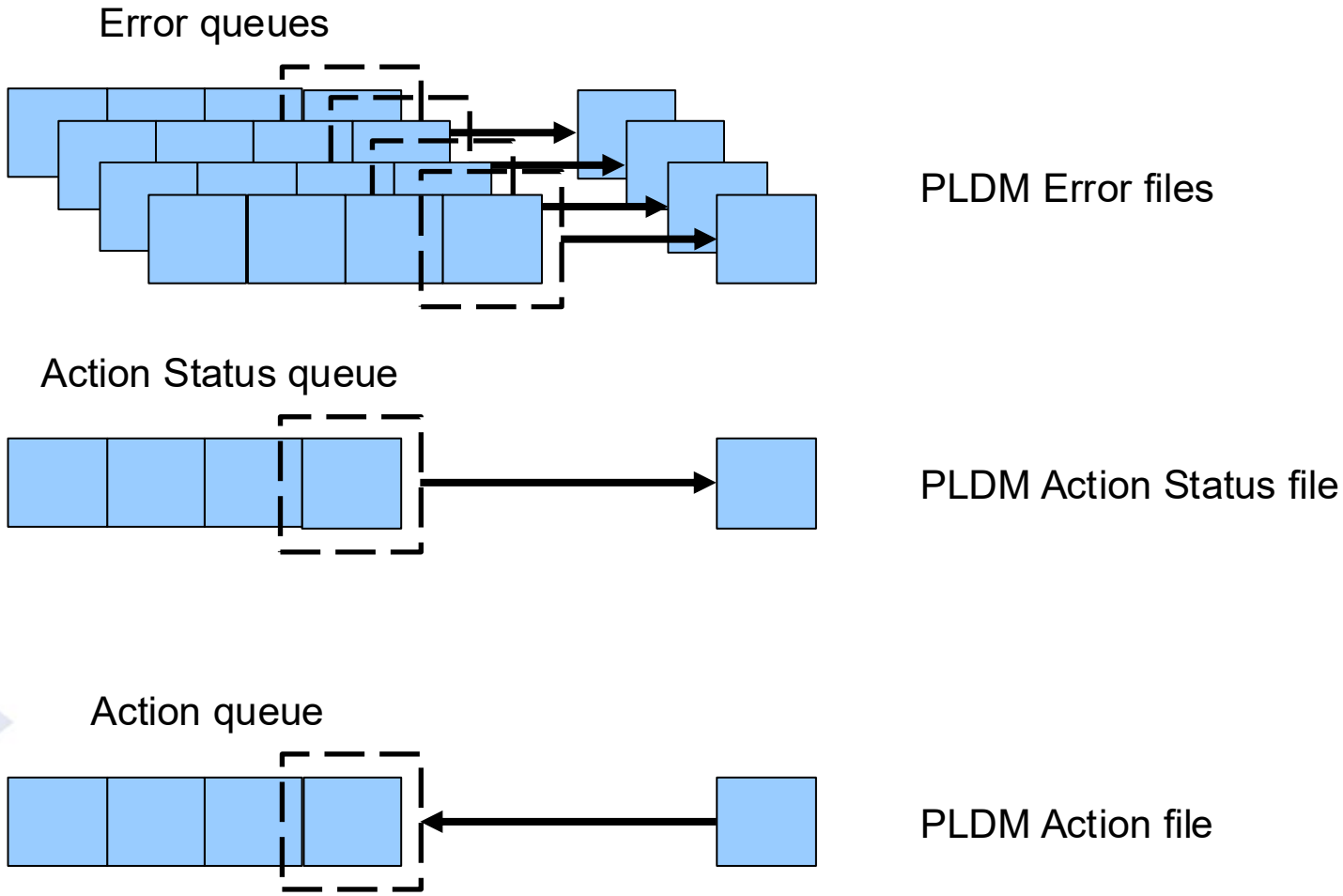
RAS API model diagram



RAS API EP model

- RAS API EP has top-level structured attribute describing:
  - PlatformID (EP unique identifier)
  - DSP2094 version EP is designed against.
- CPER and CPAD files are a front for CPER and CPAD queues.
  - SerialRxFifo / SerialTxFifo potential alternative -- PLDM file represent whole queue, with additional complications (e.g. how to separate CPER records within the stream?)
- Up to 4 CPER queues, with the following severities:
  - Info
  - Corr
  - Uncorr
  - Fatal

  A CPER queue can contain multiple severities.
- 1 Platform Action Status queue
- 1 Action queue

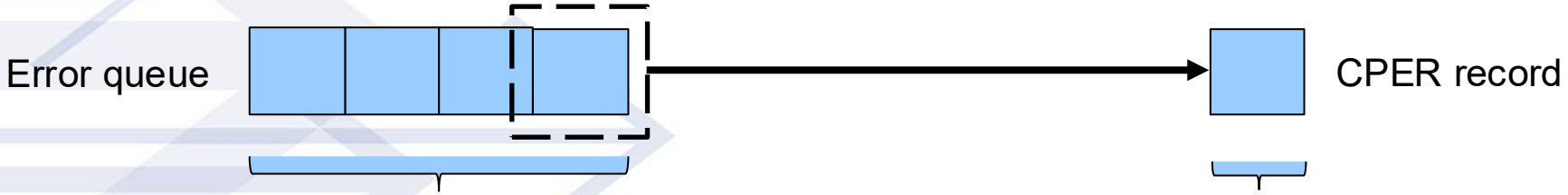


CPER Queue

- Contains CPER records that describe:
  - error records
  - action results

Commands that interact with it:

- Get_CPER (EP, queue) -> status, CPER
- [optional] Clear_CPER (EP, queue, Record_id) -> status
  - If not supported, Get_CPER implicitly clears record read
- If Queue overflows sensor transitions to *stressed* (Set ID 9), transitions back to *normal* once next CPER record retrieved.



- Overflow State Sensor (Set ID 9)
- Clear structured attribute
- Queue property structured attribute

- File Size Sensor
- File State Sensor

Action Queue

- Contains CPAD records that describe Action requests
  - An Action will generate an *action status* result, communicated as a CPER record

Commands that interact with it:

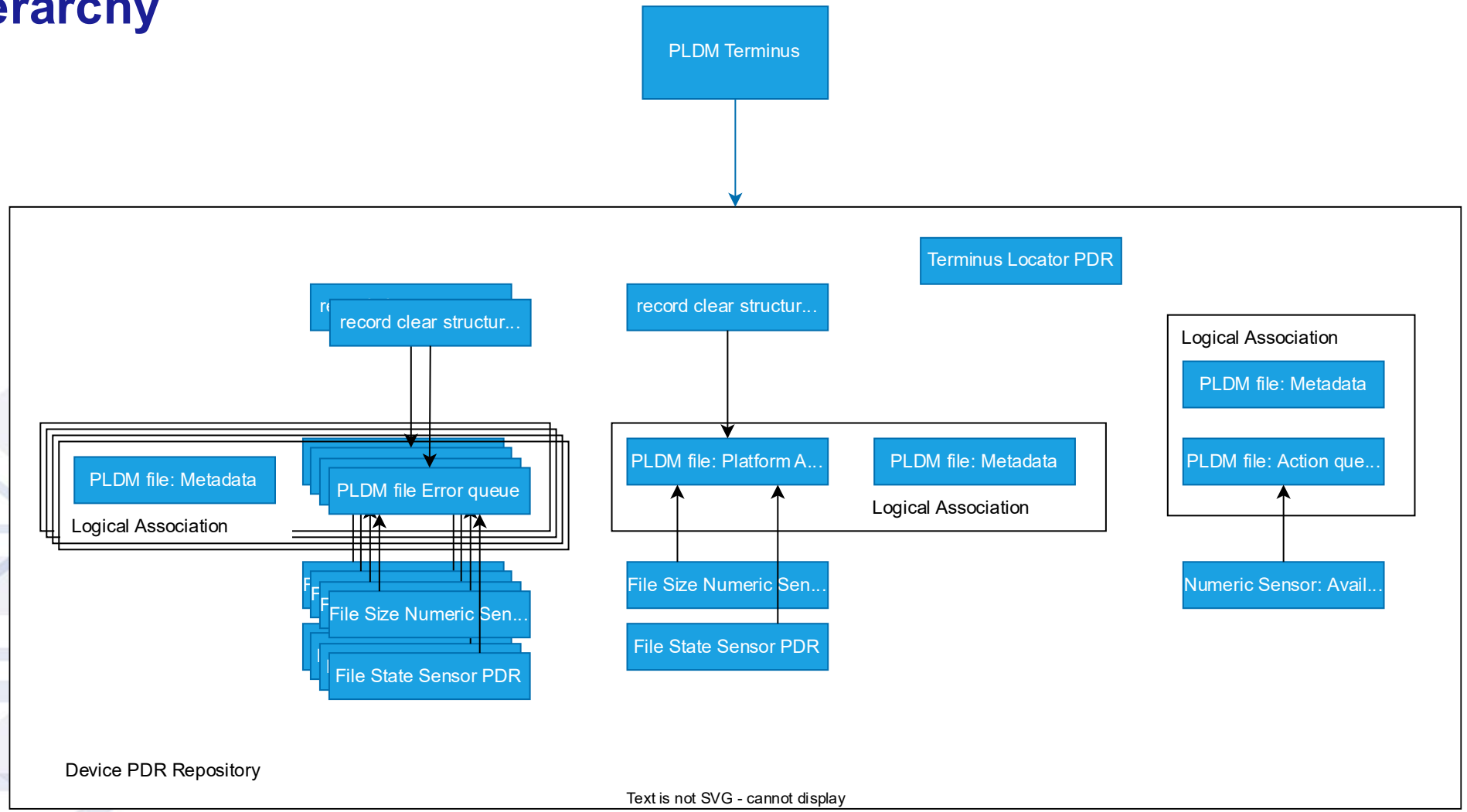
- Put_CPAD (EP, CPAD) -> status

- Put_CPAD fails if *Free Entries Count* == 0



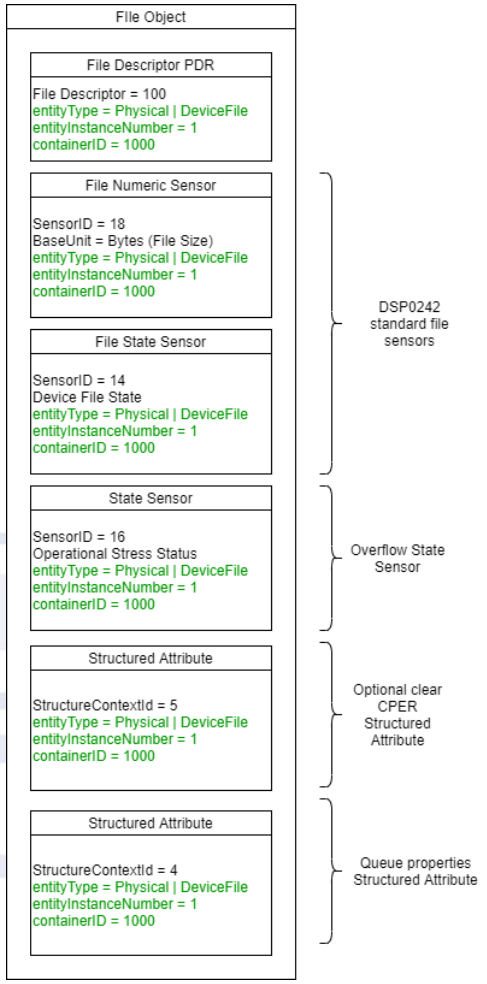
PDRS

PDR Hierarchy

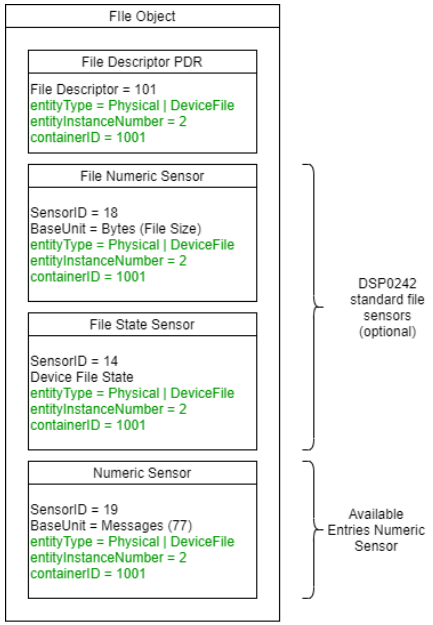


PDR structures

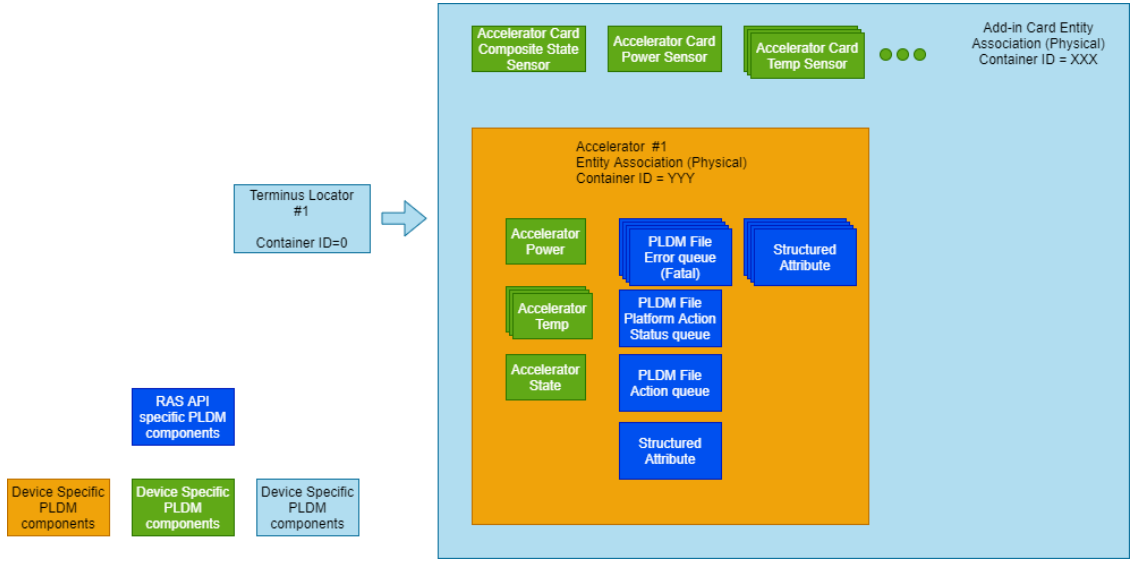
CPER Queue



Action Queue



Topology Example



QUESTIONS

BACKUP

PLDM type 7 SerialTxFifo semantics

- DfRead is single transfer part per section and single section
  - XFER_FIRST_PART always followed by XFER_COMPLETE
  - File Host clears info read (by advancing file pointer)

- File Client must keep track of start/finish of CPER records, which means either:
  - Client inspects the CPER record to discover length, or
  - PLDM mapping defines escape characters (frail: actual CPER data can be confused with character)

- A CPER record could be partially transferred:
  - E.g. CPER is larger than *section* size, or section alignment leads to part of CPER transferred
  - CPER record partially in File Host (top part) and Client (bottom part) – is this a problem?

RAS API defined commands

All commands invoked by RAS API Initiator

- Get_CPER (EP, queue) -> status, CPER
- Clear_CPER (EP, queue, Record_id) -> status – optional command, when not implemented, Get_CPER has "*pop*" semantics.
- Put_CPAD (EP, CPAD) -> status
- Get_EP_config (EP) -> status, EP_config
- Write_EP_config (EP, EP_config) -> status